Cuprins

1 Preprocesare imagini si concepte de baza **2**
1.1 Conversie grayscale 2
1.2 Conversie HSV 2
1.3 Masca de culoare (threshold pe interval) 2
1.4 Filtrare bilaterala 3

2 Edge detection si segmentare **4**
2.1 Thresholding adaptiv Otsu 4
2.2 Canny Edge Detection 5
2.3 Bloc reduce / downsampling 6

3 Regresie liniara **7**
3.1 Concept general si utilitate 7
3.2 Metoda celor mai mici patrate 7
3.3 Interpretarea parametrilor 7
3.4 Filtrarea punctelor si stabilizarea fit-ului 8
3.5 Rezultate si performanta 8

4 Concepte suplimentare **9**
4.1 Evaluarea stabilitatii fit-ului 9
4.2 Predictia urmatorului cadru 9
4.3 Region of Interest (ROI) 9
4.4 Scaling uniform 9

1 Preprocesare imagini si concepte de baza

1.1 Conversie grayscale

Conversia unei imagini color la grayscale reduce complexitatea datelor, pastrand doar informatia de intensitate luminoasa. Practic, fiecare pixel devine un scalar proportional cu perceptia luminii, ceea ce permite algoritmilor de procesare sa lucreze mai rapid. Intern, se combina valorile RGB folosind ponderi care reflecta perceptia vizuala a fiecarui canal, fara a afecta structurile importante, cum sunt marginile. De exemplu, pentru un pixel P cu valorile R, G, B:

$$P = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$$

1.2 Conversie HSV

Modelul HSV separa culoarea (Hue) de saturatie (S) si luminozitate (V), permitand operatii independente pe culoare sau intensitate. Intern, aceasta inseamna transformarea din sistemul RGB in coordonate polare pentru H si scalare pentru S si V. Fie un pixel cu componente (R, G, B) normalizate intre 0 și 1. Valorile HSV (H, S, V) se definesc astfel:

$$V = \max(R, G, B)$$

$$C = V - \min(R, G, B)$$

$$S = \begin{cases} 0, & \text{daca } V = 0 \\ \frac{C}{V}, & \text{altfel} \end{cases}$$

$$H = \begin{cases} 0, & \text{daca } C = 0 \\ 60^\circ \cdot \frac{G-B}{C} \mod 360^\circ, & \text{daca } V = R \\ 60^\circ \cdot \frac{B-R}{C} + 120^\circ, & \text{daca } V = G \\ 60^\circ \cdot \frac{R-G}{C} + 240^\circ, & \text{daca } V = B \end{cases}$$

Utilitatea practica este filtrarea precisa a nuantelor de interes indiferent de luminozitatea regiunii.

1.3 Masca de culoare (threshold pe interval)

Masca de culoare selecteaza doar pixelii care se afla intr-un anumit interval de valori HSV. Teoretic, se construieste o functie indicator $M(x, y)$ care ia valoarea 1 daca pixelul respecta criteriile si 0 altfel. Intern, fiecare pixel este comparat cu limitele inferioare si superioare pentru fiecare componenta, rezultand o imagine binara care delimiteaza regiunile de interes. Aceasta separare simplifica urmatoarele etape de procesare si reduce zgomotul provenit din alte regiuni:

$$M(x, y) = \begin{cases} 1 & \text{daca} \quad \text{lower} \leq HSV(x, y) \leq \text{upper} \\ 0 & \text{altfel} \end{cases}$$

1.4 Filtrare bilaterala

Filtrarea bilaterala reduce zgomotul pastrand marginile. Conceptul teoretic combina doua componente: influenta spatiala (distanta intre pixeli) si influenta de intensitate (diferenta de valoare a pixelilor). Pentru un pixel x, avem pixelii y din vecinatatea Ω:

$$I_{\text{blur}}(x) = \frac{1}{W_p} \sum_{y \in \Omega} I(y) \exp\left(-\frac{|x-y|^2}{2\sigma_s^2}\right) \exp\left(-\frac{|I(x)-I(y)|^2}{2\sigma_r^2}\right)$$

unde W_p este factorul de normalizare, σ_s controleaza cat de mult sunt luati in considerare pixelii apropiati spatial, iar σ_r controleaza cat de mult influenteaza valorile apropiate in intensitate. Intern, filtrul compara fiecare pixel cu vecinii sai, acordand mai multa greutate celor apropiati spatial si de culoare, astfel reducand zgomotul dar pastrand discontinuitatile, cum sunt marginile obiectelor, esential pentru detectia orizontului.

Figure 1: Stanga - imagine originala, dreapta - imagine filtrata.

2 Edge detection si segmentare

2.1 Thresholding adaptiv Otsu

Thresholding-ul Otsu separa pixelii unei imagini in doua clase (de exemplu cer si teren) prin alegerea unui prag T care maximizeaza dispersia intre clase. Aceasta metoda asigura ca pixelii din fiecare clasa sunt cat mai omogeni, iar diferentele intre clase sunt cat mai mari.

Notatii si variabile:

- L: numarul de nivele de intensitate / numarul de valori posibile pentru R, G sau B (de obicei 256 pentru imagini pe 8 biti)
- n_i: numarul de pixeli cu intensitatea i
- N: numarul total de pixeli
- $p_i = n_i/N$: probabilitatea ca un pixel sa aiba intensitatea i
- $w_0(T), w_1(T)$: ponderile pixelilor sub si peste pragul T
- $\mu_0(T), \mu_1(T)$: mediile intensitatilor pentru cele doua clase

Ponderile si mediile claselor pentru un prag T:

$$w_0(T) = \sum_{i=0}^{T} p_i, \quad w_1(T) = \sum_{i=T+1}^{L-1} p_i$$

$$\mu_0(T) = \frac{\sum_{i=0}^{T} i\, p_i}{w_0(T)}, \quad \mu_1(T) = \frac{\sum_{i=T+1}^{L-1} i\, p_i}{w_1(T)}$$

Dispersia intre clase:

$$\sigma_b^2(T) = w_0(T) w_1(T) (\mu_0(T) - \mu_1(T))^2$$

Algoritm pas cu pas:

1. Pentru fiecare prag posibil $T \in [0, L-1]$, se calculeaza:
   - Ponderile $w_0(T), w_1(T)$
   - Mediile $\mu_0(T), \mu_1(T)$
2. Se evalueaza dispersia intre clase $\sigma_b^2(T)$ folosind formula de mai sus
3. Se selecteaza pragul T care maximizeaza $\sigma_b^2(T)$

Binarizare: imaginea finala I_{bin} se obtine prin compararea fiecarui pixel cu pragul T:

$$M(x,y) = \begin{cases} 1 & \text{daca} \quad \text{I(x,y)} > T \\ 0 & \text{altfel} \end{cases}$$

2.2 Canny Edge Detection

Algoritmul Canny identifica marginile semnificative intr-o imagine, pastrand doar discontinuitatile reale si eliminand zgomotul. Este utilizat pe imagini grayscale si functioneaza in patru etape principale:

1. Filtrare Gaussiana (reducerea zgomotului)

Zgomotul din imagine poate crea false margini. Se aplica un filtru Gaussian G_σ pentru a netezi imaginea:

$$I_{\text{smooth}}(x,y) = (I * G_\sigma)(x,y) = \sum_{u=-k}^{k} \sum_{v=-k}^{k} I(x-u, y-v)\, G_\sigma(u,v)$$

unde $G_\sigma(u,v) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{u^2+v^2}{2\sigma^2}\right)$ si k determina dimensiunea kernelului (ex. $k = 1$ pentru kernel 3×3).

Exemplu concret kernel 3×3 cu $\sigma = 1$:

$$G_1 = \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Aplicand acest kernel, fiecare pixel din I_{smooth} devine media ponderata a vecinatatii sale, reducand zgomotul dar pastrand tendintele generale ale intensitatilor.

2. Calcul gradient

Se estimeaza derivata pe axele x si y pentru fiecare pixel:

$$G_x = \frac{\partial I_{\text{smooth}}}{\partial x}, \quad G_y = \frac{\partial I_{\text{smooth}}}{\partial y}$$

Magnitudinea gradientului si directia acestuia sunt:

$$|G| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan \frac{G_y}{G_x}$$

$|G|$ indica cat de abrupt se schimba intensitatea (posibila margine), iar θ indica orientarea marginii

3. Non-maximum suppression (subtierea marginilor)

Se pastreaza doar pixelii care reprezinta maxime locale in directia gradientului θ. Astfel, liniile devin subțiri (un singur pixel latime).

4. Thresholding (numit si hysteresis la acest algoritm)

Se definesc doua praguri: T_{low} si T_{high}.

- Pixelii cu $|G| > T_{\text{high}}$ sunt considerati marginile sigure
- Pixelii cu $T_{\text{low}} < |G| \leq T_{\text{high}}$ sunt pastrati doar daca sunt conectati la marginile sigure

Aceasta metoda elimina punctele izolate si pastreaza doar marginile continue.

Observatii: - Alegerea lui σ influenteaza cat de mult se netezeste imaginea. Dimensiunea kernelului trebuie sa fie suficient de mare pentru zgomot, dar nu exagerat, altfel marginile se estompeaza. Pragurile T_{low} si T_{high} controleaza sensibilitatea detectorului

2.3 Bloc reduce / downsampling

Downsampling-ul reduce dimensiunea imaginilor pentru a creste performanta algoritmilor de procesare, pastrand informatia esentiala. Aceasta operatie este utila in special in detectia marginilor sau segmentarea imaginii, unde procesarea fiecarei unitati de pixel poate fi costisitoare.

Concept si notatii:

- Fie imaginea originala I de dimensiune $H \times W$.
- Imaginea se imparte in blocuri de dimensiune $k \times k$, unde fiecare bloc contine k^2 pixeli vecini.
- Fiecare bloc B este redus la o singura valoare printr-o functie de agregare:
  - **Media blocului:**
  $$I_{\text{down}}(i,j) = \frac{1}{k^2} \sum_{(u,v)\in B} I(u,v)$$
  - **Maxim local:**
  $$I_{\text{down}}(i,j) = \max_{(u,v)\in B} I(u,v)$$
- (i,j) indica coordonatele pixelului in imaginea downsamplata, (u,v) coordonatele in blocul original.

Algoritm:

1. Se imparte imaginea originala in blocuri $k \times k$.
2. Pentru fiecare bloc, se aplica functia de agregare (media sau maximul).
3. Rezultatul este imaginea I_{down} de dimensiune aproximativ $H/k \times W/k$, redusa proportional.

Prin algoritm se reduce numarul total de pixeli de procesar, deci ce va urma ulterior se va rula mai repede. Totodata, se pastreaza caracteristicile majore, cum sunt marginile, pentru ca agregarea nu elimina contrastul local. Parametrul k controleaza compromisul intre viteza si rezolutia detaliilor: un k mai mare creste viteza dar pierde detalii fine, in timp ce un k mai mic pastreaza mai multa informatie.

3 Regresie liniara

Regresia liniara este folosita pentru a aproxima o relatie liniara intre doua variabile, in cazul nostru coordonatele x si y ale punctelor detectate pe conturul liniei orizontului. Se utilizeaza metoda celor mai mici patrate, care minimizeaza suma patratelor erorilor verticale intre punctele reale si linia de fit.

3.1 Concept general si utilitate

Regresia liniara transforma un set de puncte discrete intr-o reprezentare continua sub forma unei linii $y = mx + b$. Aceasta linie de fit:

- Captureaza directia generala a datelor (ex. inclinatia orizontului)
- Reduce efectul punctelor izolate sau eronate
- Permite calculul unor parametri derivati, cum ar fi unghiurile Roll si Pitch
- Este foarte eficienta din punct de vedere al performantei, necesitand doar operatii algebrice simple (sume, inmultiri, impartiri)

In general, regresia liniara este utila oriunde dorim sa extragem o tendinta principala dintr-un set de date, sa facem predictii sau sa filtram zgomotul.

3.2 Metoda celor mai mici patrate

Fie n puncte (x_i, y_i), $i = 1, \ldots, n$. Linia are forma:

$$y = mx + b$$

Functia de eroare: minimizam suma patratelor distantei verticale intre puncte si fit:

$$E(m, b) = \sum_{i=1}^{n} (y_i - mx_i - b)^2$$

Solutia analitica se obtine derivand E fata de m si b si rezolvand sistemul:

$$m = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}, \quad b = \bar{y} - m\bar{x}$$

3.3 Interpretarea parametrilor

- m - panta liniei: indica inclinatia orizontului fata de orizontala imaginii
- b - intersectia cu axa y: pozitia verticala a liniei
- $\bar{x}, \bar{y}$ - mediile coordonatelor punctelor

3.4 Filtrarea punctelor si stabilizarea fit-ului

In procesarea secventelor de imagini, se poate folosi predictia liniei anterioare pentru a elimina punctele care deviaza prea mult de la fit-ul precedent. Aceasta utilizeaza pragul *exclusion threshold*:

- Se estimeaza linia $y = m_{\text{prev}}x + b_{\text{prev}}$
- Se calculeaza distanta perpendiculara de la fiecare punct la linia prezisa
- Se elimina punctele care depasesc pragul, pastrand doar punctele consistente

Astfel putem:

- Reduce erorile introduse de punctele zgomotoase
- Creste stabilitatea liniei de fit pe mai multe cadre
- Permite estimari mai precise

3.5 Rezultate si performanta

Regresia liniara este rapida si robusta:

- Complexitatea este liniara in numarul de puncte n
- Se obtine o linie de fit chiar si in prezenta punctelor eronate
- Este usor de integrat in pipeline-uri
- Ofera o reprezentare continua si parametri cuantificabili

4 Concepte suplimentare

4.1 Evaluarea stabilitatii fit-ului

Varianta este folosita pentru a cuantifica cat de bine se potrivesc punctele detectate pe linia de fit. Fie n puncte (x_i, y_i) si linia $y = mx + b$, distanta perpendiculara de la un punct la linie se noteaza d_i. Dispersia medie este:

$$\text{variance} = \frac{1}{n} \sum_{i=1}^{n} d_i^2$$

- Varianta mica $\rightarrow$ fit-ul este stabil si liniaritatea punctelor este buna
- Varianta mare $\rightarrow$ puncte dispersate $\rightarrow$ detectia poate fi nesigura
- Pragul *acceptable_variance* stabileste limita de incredere a liniei

4.2 Predictia urmatorului cadru

Se poate folosi o predictie simpla bazata pe extrapolare liniara a unghiurilor:

$$\text{Roll}_{t+1} = \text{Roll}_t + (\text{Roll}_t - \text{Roll}_{t-1})$$

$$\text{Pitch}_{t+1} = \text{Pitch}_t + (\text{Pitch}_t - \text{Pitch}_{t-1})$$

Aceasta filtrare ajuta la reducerea zgomotului si la stabilizarea detectiei intre cadre consecutive.

4.3 Region of Interest (ROI)

ROI reprezinta zona relevanta a imaginii (ex. cerul), care este decupata si scalata pentru a mentine aspect ratio-ul. Aceasta selectie:

- Reduce numarul de pixeli de procesat
- Imbunatateste acuratetea algoritmilor de detectie
- Evita distorsiuni cauzate de redimensionare neuniforma

4.4 Scaling uniform

Ajustarea rezolutiei imaginilor pentru consistenta intre cadre se realizeaza prin scalare uniforma:

$$I_{\text{scaled}} = \text{resize}(I_{\text{ROI}}, s)$$

unde s este factorul de scalare calculat pentru a pastra raportul de aspect. Aceasta asigura ca toate cadrele au aceeasi dimensiune relativa, permitand aplicarea algoritmilor de filtrare, regresie si detectie fara ajustari suplimentare.